Training deep networks, softmax, and regularization

CSI 4106 - Fall 2026

Marcel Turcotte
Version: Oct 17, 2026 16:03

Preamble

Message of the Day

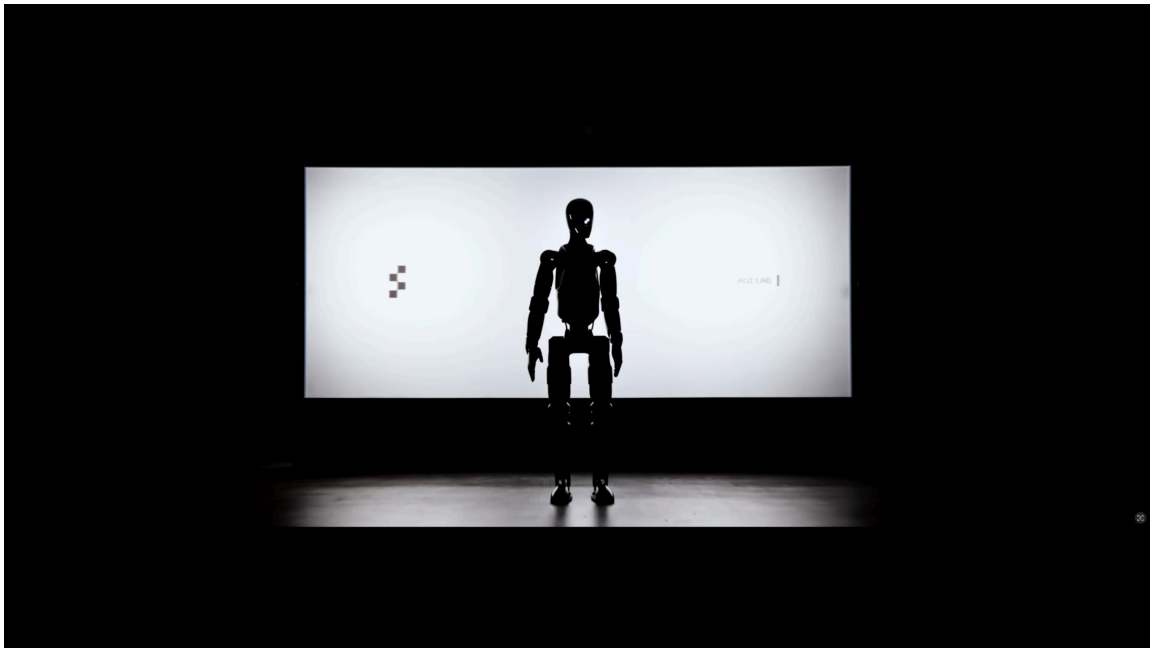
https://youtu.be/0Ge3jKLDJaA

The collie and the pug | HackMIT keynote 2026, Posting on YouTube on 2026-09-24.

> **Note**
>
> Grant Sanderson explores the evolving relationship between technical understanding and the act of building in modern software development. By comparing the motivations of different approaches, this keynote encourages attendees to reflect on their own goals for learning and creation during the event.

Grant Sanderson is the creator of 3Blue1Brown, a YouTube channel known for exceptionally clear, visual explanations of mathematics. His work helped popularize intuitive, animation-driven math education online.

Message of the Day



Why 2025 is the single most pivotal year in our lifetime, Peter Leyden, Big Think, YouTube, 2025-10-20.

Learning objectives

- **Explain** how activation functions and initialization affect gradient propagation.
- **Compare** `SimplerMLP` with its extension using activation choices, L2, and learning-rate decay.
- **Explain** the role of softmax and cross-entropy loss in measuring the dissimilarity between predicted and true probability distributions.
- **Explore** methods like L1, L2 regularization, and dropout to enhance neural networks' generalization capabilities.

Lecture 13 implemented backpropagation in `SimplerMLP`, using sigmoid units, binary cross-entropy, and a constant learning rate. We first examine gradient propagation and initialization, then extend that implementation as `SimpleMLP`. The remaining sections consider output activations, multiclass cross-entropy, and regularization.

Gradient propagation and initialization

Vanishing gradients

- Backpropagation repeatedly multiplies **weights and local activation derivatives**.
- Small factors can make early-layer gradients too small for effective learning.

- Saturated sigmoid units have derivatives close to zero.

Recall $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq 0.25$.

Lecture 13's scalar example contained one hidden derivative. A deeper network introduces more such factors along each path. The sigmoid derivative approaches zero for large positive or negative preactivations, so saturated units can strongly attenuate the signal propagated upstream.

Weights also enter these products and different paths contribute sums: the bound of 0.25 on a sigmoid's local derivative is not a bound on every complete parameter gradient. Gradients can shrink or grow depending on the combined transformations. Both vanishing and exploding gradients can occur in feed-forward as well as recurrent networks.

For sigmoid output with binary cross-entropy, the output error signal simplifies to $\hat{p} - y$. The explicit sigmoid factor cancels there, but hidden sigmoid units still contribute their activation derivatives. This is why the simplified output formula does not remove the hidden-layer problem.

Activation functions and their derivatives

In [2]:
```
# Attribution: https://github.com/ageron/handson-ml3/blob/main/10_neural_net

import numpy as np
import matplotlib.pyplot as plt

from scipy.special import expit as sigmoid

def relu(z):
    return np.maximum(0, z)

def derivative(f, z, eps=0.000001):
    return (f(z + eps) - f(z - eps))/(2 * eps)

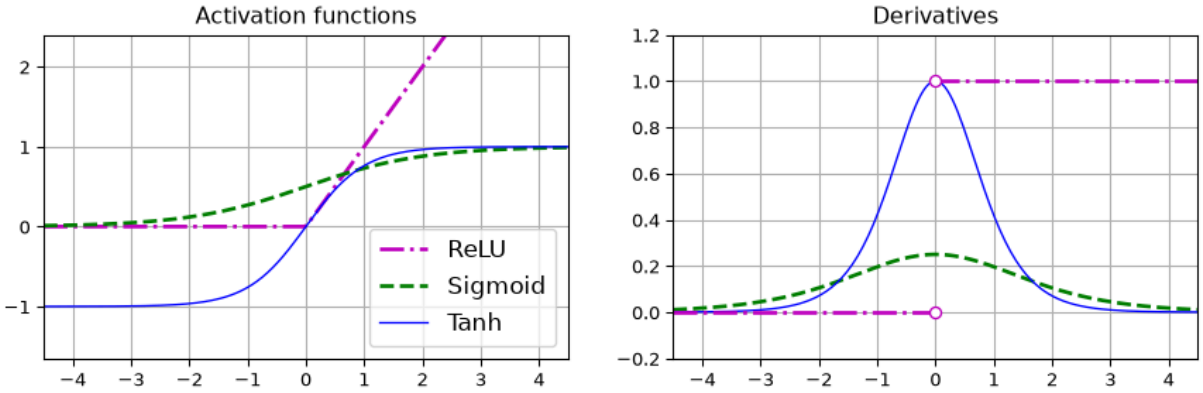
max_z = 4.5
z = np.linspace(-max_z, max_z, 200)

plt.figure(figsize=(11, 3.1))

plt.subplot(121)
plt.plot(z, relu(z), "m-.", linewidth=2, label="ReLU")
plt.plot(z, sigmoid(z), "g--", linewidth=2, label="Sigmoid")
plt.plot(z, np.tanh(z), "b-", linewidth=1, label="Tanh")
plt.grid(True)
plt.title("Activation functions")
plt.axis([-max_z, max_z, -1.65, 2.4])
plt.gca().set_yticks([-1, 0, 1, 2])
plt.legend(loc="lower right", fontsize=13)
```

```
plt.subplot(122)
plt.plot(z, derivative(sigmoid, z), "g--", linewidth=2, label="Sigmoid")
plt.plot(z, derivative(np.tanh, z), "b-", linewidth=1, label="Tanh")
plt.plot([-max_z, 0], [0, 0], "m-.", linewidth=2)
plt.plot([0, max_z], [1, 1], "m-.", linewidth=2)
plt.plot([0, 0], [0, 1], "mo", markersize=6, markerfacecolor="white")
plt.grid(True)
plt.title("Derivatives")
plt.axis([-max_z, max_z, -0.2, 1.2])

plt.show()
```



Géron (2022) – 10_neural_nets_with_keras.ipynb

Activation choice affects gradient propagation and therefore training. You need to interpret these slopes and how the chain rule uses them, rather than manually derive every expression.

Sigmoid's maximum derivative is 0.25, and its derivative approaches zero at both extremes. The $\tanh$ derivative has maximum one at zero and also approaches zero in its saturated tails.

For ReLU, the derivative is zero for negative preactivations and one for positive preactivations. At zero the ordinary derivative is undefined; implementations choose a value by convention, commonly zero. The open circles mark the missing ordinary derivative. The activation itself can take any nonnegative value: it is the derivative, away from zero, that takes values zero or one.

ReLU produces sparse activations because it outputs zero for nonpositive inputs. A zero activation for one example is normal. A unit that remains in this region across the relevant training inputs receives no data-loss gradient through its activation and may remain inactive: the dying ReLU problem. This is persistent inactivity, not simply one zero output. Changes in upstream representations can sometimes reactivate a unit, so inactivity is not invariably irreversible. See Lu et al. (2019).

ReLU's positive-side derivative avoids one source of attenuation, but does not make the entire gradient equal to one. Incoming activations, downstream weights, other activation

derivatives, and path sums still matter. Leaky ReLU retains a small negative-side slope, reducing this particular risk.

Vanishing gradients: solutions

- **Alternative activation functions**: Rectified Linear Unit (**ReLU**) and its variants (e.g., **Leaky ReLU**, **Parametric ReLU**, and **Exponential Linear Unit**).
- **Weight Initialization**: Xavier (Glorot) or He initialization.

Activation functions and initialization influence the propagation of signals in both directions. ReLU avoids saturation for positive preactivations, but its zero derivative on the negative side creates a different limitation, discussed next. Choosing ReLU does not guarantee stable gradients in an arbitrarily deep network. Normalization layers and residual connections provide other approaches beyond this worked implementation.

Glorot and Bengio

Figure 6

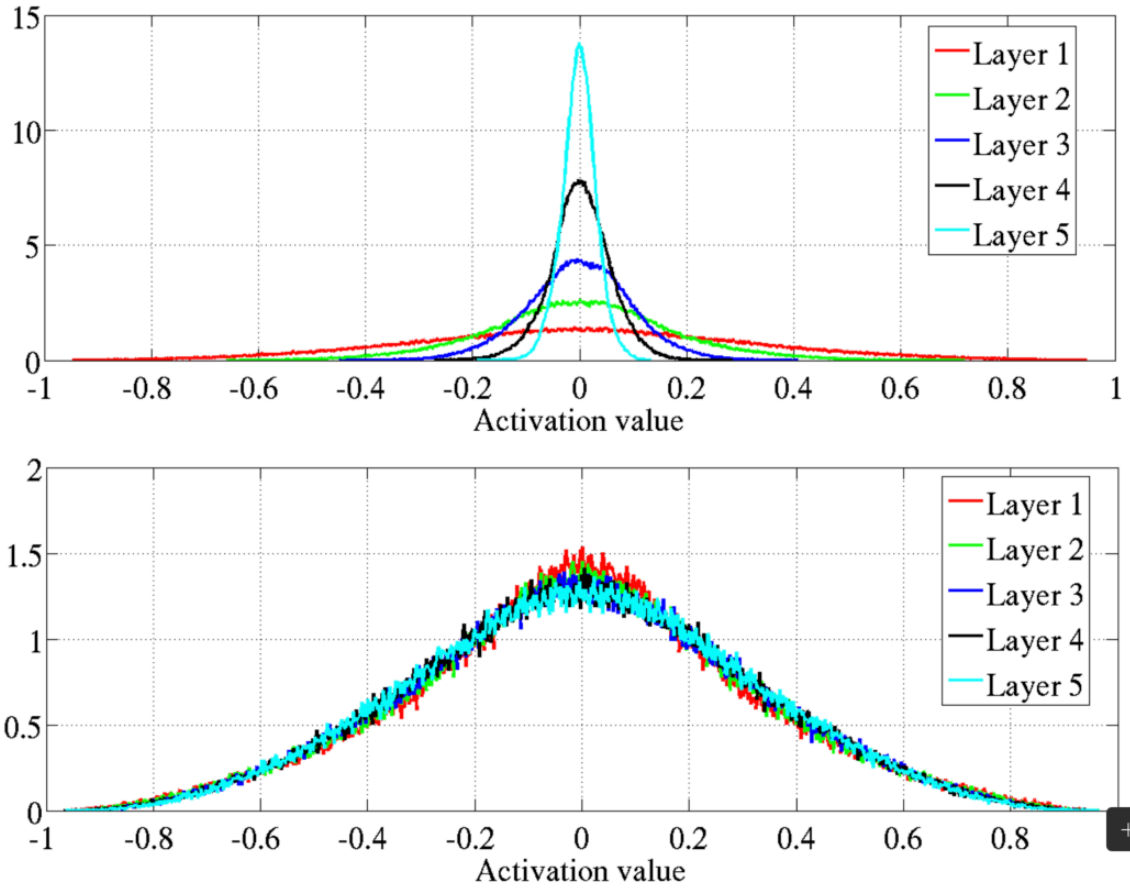
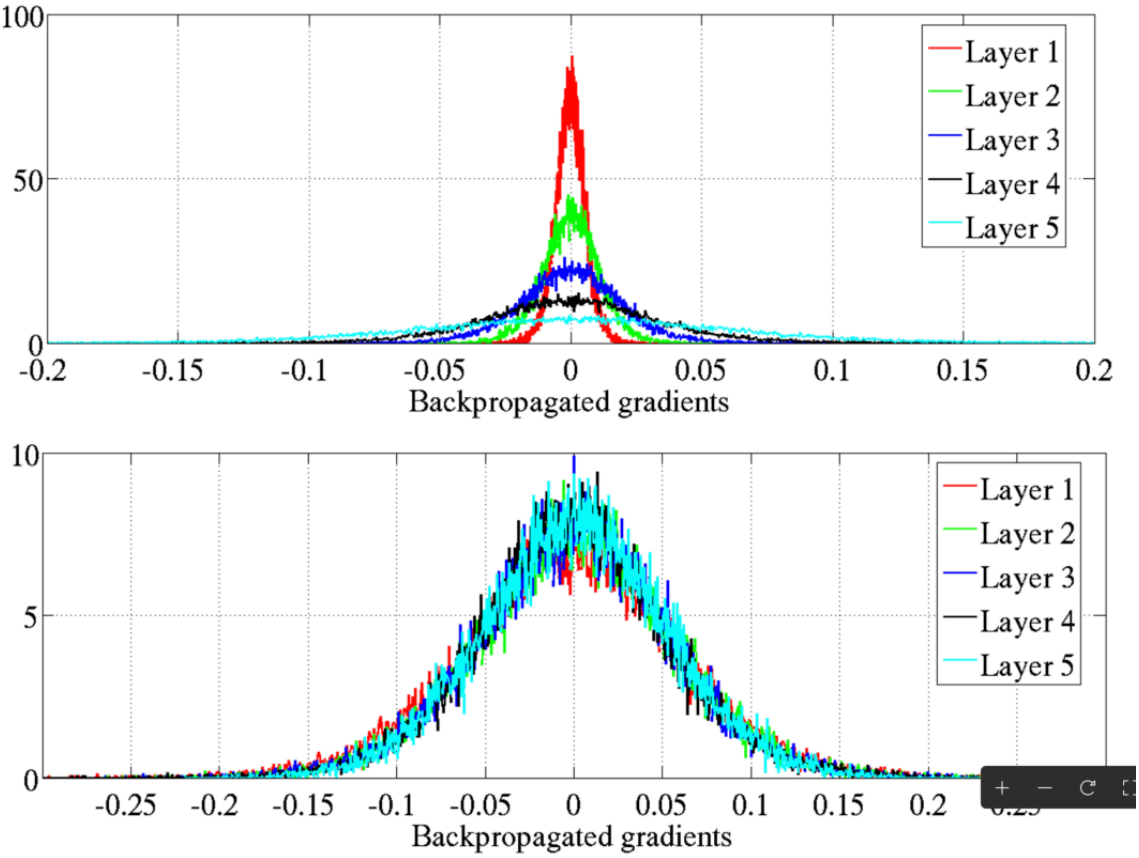


Figure 7



Glorot and Bengio (2010), page 254.

These figures compare standard initialization (top) with the normalized initialization proposed by Glorot and Bengio (bottom), for a network with $\tanh$ units (Glorot and Bengio 2010, 254). Figure 6 shows activation histograms; Figure 7 shows back-propagated-gradient histograms. With the standard initialization, activations become more concentrated near zero in later layers, while back-propagated gradients are more concentrated near zero closer to the input. Normalized initialization reduces the discrepancy across layers in these experiments.

These are distributions for the reported setup, not a universal rule that vanishing gradients always imply small forward activations. A saturated sigmoid can have an activation near one and a derivative near zero. The paper also distinguishes back-propagated signals from weight gradients; their layerwise distributions need not behave identically.

The Xavier/Glorot normal initializer balances input and output layer widths using $\mathrm{Var}(W) = 2/(n_{\text{in}} + n_{\text{out}})$. It is the scaled random initialization already used by `SimplerMLP`.

He initialization

For ReLU hidden layers, He normal initialization uses

$$\mathrm{Var}(W) = \frac{2}{n_{\text{in}}}.$$

```
def he_init(rng, fan_in, fan_out):
    std = np.sqrt(2.0 / fan_in)
    return rng.normal(0.0, std, size=(fan_in, fan_out))
```

Also called Kaiming initialization.

The scaling accounts for the rectifier's effect on signal variance. `SimpleMLP` will use He initialization for ReLU hidden layers and Xavier/Glorot initialization for sigmoid layers, including the output layer. These are initialization rules, not guarantees about the entire training trajectory. See He et al. (2015) for the rectifier analysis.

Note

Randomly initializing the weights[1] is sufficient to **break symmetry** in a neural network, **allowing the bias terms to be set to zero** without impacting the network's ability to learn effectively.

Activation Function: Leaky ReLU

[1] Proper initialization of weights, such as using Xavier/Glorot or He initialization, is crucial and should be aligned with the choice of activation function to ensure optimal network performance.

In [3]:
```
import numpy as np
import matplotlib.pyplot as plt

# Define the Leaky ReLU function
def leaky_relu(x, alpha=0.2):
    return np.where(x > 0, x, alpha * x)

# Define the derivative of the Leaky ReLU function
def leaky_relu_derivative(x, alpha=0.2):
    return np.where(x > 0, 1, alpha)

# Generate a range of input values
x_values = np.linspace(-4, 4, 400)

# Compute the Leaky ReLU and its derivative
leaky_relu_values = leaky_relu(x_values)
leaky_relu_derivative_values = leaky_relu_derivative(x_values)

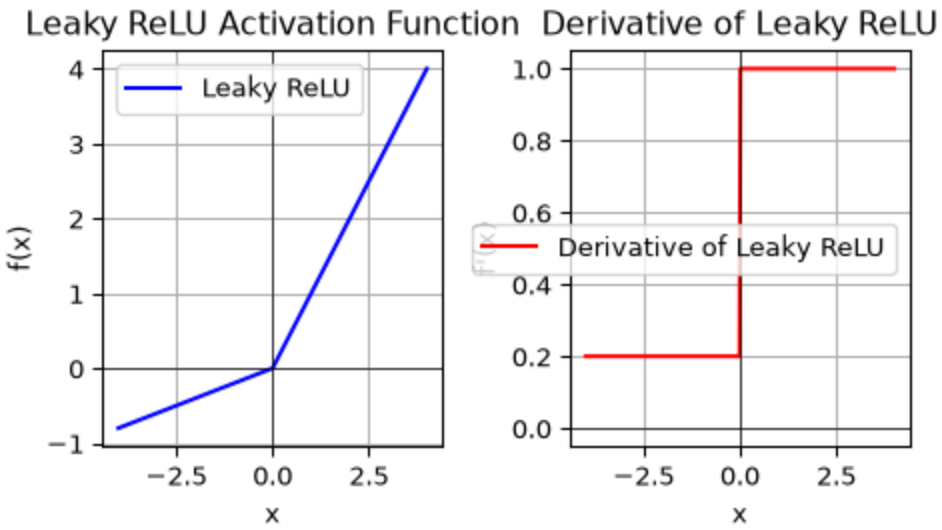
# Create the plot
plt.figure(figsize=(5, 3))

# Plot the Leaky ReLU
plt.subplot(1, 2, 1)
plt.plot(x_values, leaky_relu_values, label='Leaky ReLU', color='blue')
```

```
plt.title('Leaky ReLU Activation Function')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.grid(True)
plt.axhline(0, color='black',linewidth=0.5)
plt.axvline(0, color='black',linewidth=0.5)
plt.legend()

# Plot the derivative of the Leaky ReLU
plt.subplot(1, 2, 2)
plt.plot(x_values, leaky_relu_derivative_values, label='Derivative of Leaky
plt.title('Derivative of Leaky ReLU')
plt.xlabel('x')
plt.ylabel("f'(x)")
plt.grid(True)
plt.axhline(0, color='black',linewidth=0.5)
plt.axvline(0, color='black',linewidth=0.5)
plt.legend()

# Show the plots
plt.tight_layout()
plt.show()
```



The Leaky ReLU, a variant of the standard ReLU activation function, effectively mitigates the issue of *dying ReLU nodes*. For negative input values, it introduces a linear component with a slope governed by the parameter `negative_slope`.

When the input to the ReLU activation function, the weighted sum plus bias, is negative for all the training examples, the output value of ReLU is zero. But also, its derivative is 0, which effectively deactivates the neuron. Leaky ReLU, or other variants, effectively mitigates the issue.

```
import tensorflow as tf
from tensorflow.keras.layers import Dense

leaky_relu = tf.keras.layers.LeakyReLU(negative_slope=0.2)
```

```
dense = tf.keras.layers.Dense(50, activation=leaky_relu,
kernel_initializer="he_normal")
```

See the Keras activation functions for other choices.

Extending the implementation

From SimplerMLP to SimpleMLP

Component	`SimplerMLP` (Lecture 13)	`SimpleMLP`
Hidden activation	Sigmoid	Sigmoid or ReLU
Output	Sigmoid probability	Sigmoid probability
Learning rate	Constant	Constant or exponential decay
Loss	Mean BCE	Mean BCE plus optional L2
Example order	Shuffle every epoch	Optional shuffling (default: on)
Caches and backward loop	Same structure	Same structure
Prediction and training API	Same methods	Same methods

The new class extends the previous implementation while retaining its computational structure. Both are binary classifiers with one sigmoid output. Changing the hidden activation requires a matching local derivative and suitable initialization, but the chain rule and backward loop remain the same.

Setting `hidden_activation="sigmoid"`, `l2=0.0`, and `lr_decay=None` recovers the `SimplerMLP` computation. With the same seed and the default `shuffle=True`, the two classes initialize and shuffle identically. `SimpleMLP` adds a `shuffle` option to `train`: setting `shuffle=False` preserves the supplied example order across epochs. This is useful when training depends on that order, as discussed in Lecture 13; it does not by itself make this feedforward classifier a sequence model. The complete definition below is included for independent notebook execution; the focused excerpts emphasize only the additions.

SimpleMLP

In [4]:
```
import numpy as np

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def sigmoid_prime(z):
    a = sigmoid(z)
    return a * (1.0 - a)
```

```python
def bce_loss(y_true, y_prob, eps=1e-9):
    """Binary cross-entropy averaged over examples."""
    p = np.clip(y_prob, eps, 1 - eps)
    return -np.mean(y_true * np.log(p) + (1 - y_true) * np.log(1 - p))

def xavier_init(rng, fan_in, fan_out):
    std = np.sqrt(2.0 / (fan_in + fan_out))
    return rng.normal(0.0, std, size=(fan_in, fan_out))

def relu(z):
    return np.maximum(0.0, z)

def relu_prime(z):
    return (z > 0).astype(float)

def he_init(rng, fan_in, fan_out):
    std = np.sqrt(2.0 / fan_in)
    return rng.normal(0.0, std, size=(fan_in, fan_out))

class SimpleMLP:
    """Binary MLP with optional ReLU hidden units, L2, and learning-rate dec

    def __init__(self, layer_sizes, lr=0.1, seed=None, l2=0.0,
                 hidden_activation="relu", lr_decay=None):
        self.sizes = list(layer_sizes)
        self.lr = float(lr)
        self.base_lr = float(lr)
        self.l2 = float(l2)
        self.hidden_activation = hidden_activation
        self.lr_decay = lr_decay
        self.rng = np.random.default_rng(seed)

        self.W = []
        L = len(self.sizes) - 1
        for l, (din, dout) in enumerate(zip(self.sizes[:-1], self.sizes[1:])
            if l < L and self.hidden_activation == "relu":
                Wk = he_init(self.rng, din, dout)
            else:
                Wk = xavier_init(self.rng, din, dout)
            self.W.append(Wk)
        self.b = [np.zeros(dout) for dout in self.sizes[1:]]

    def _act(self, z, last=False):
        if last:
            return sigmoid(z)
        return relu(z) if self.hidden_activation == "relu" else sigmoid(z)

    def _act_prime(self, z, last=False):
        if last:
            return sigmoid_prime(z)
        return relu_prime(z) if self.hidden_activation == "relu" else sigmoi

    def forward(self, X):
        a = X
        L = len(self.W)
        for l, (W, b) in enumerate(zip(self.W, self.b), start=1):
```

```
            a = self._act(a @ W + b, last=(l == L))
        return a.ravel()

    def predict_proba(self, X):
        return self.forward(X)

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X) >= threshold).astype(int)

    def loss(self, X, y):
        p = self.predict_proba(X)
        base = bce_loss(y, p)
        if self.l2 > 0:
            base += 0.5 * self.l2 * sum((W ** 2).sum() for W in self.W)
        return base

    def _forward_full(self, X):
        a = X
        activations = [a]
        zs = []
        L = len(self.W)
        for l, (W, b) in enumerate(zip(self.W, self.b), start=1):
            z = a @ W + b
            a = self._act(z, last=(l == L))
            zs.append(z)
            activations.append(a)
        return activations, zs

    def train(self, X, y, epochs=30, batch_size=None, verbose=True, shuffle=
        """X: (N, d), y: (N,); batch_size=None uses the full dataset."""
        N = X.shape[0]
        idx = np.arange(N)
        B = N if batch_size is None else int(batch_size)

        for ep in range(1, epochs + 1):
            if shuffle:
                self.rng.shuffle(idx)
            if self.lr_decay is not None:
                self.lr = self.base_lr * self.lr_decay ** (ep - 1)

            base_loss = self.loss(X, y)

            for start in range(0, N, B):
                sl = idx[start:start + B]
                Xb = X[sl]
                yb = y[sl].reshape(-1, 1)
                Bsz = Xb.shape[0]

                activations, zs = self._forward_full(Xb)
                A_L = activations[-1]

                # Each row is a single example's preactivation sensitivity.
                delta = A_L - yb
                grads_W = [None] * len(self.W)
                grads_b = [None] * len(self.b)
```

```
                # Average parameter gradients over examples exactly once.
                grads_W[-1] = activations[-2].T @ delta / Bsz
                grads_b[-1] = delta.mean(axis=0)

                for k in range(len(self.W) - 2, -1, -1):
                    z = zs[k]
                    d_act = self._act_prime(z, last=False)
                    delta = (delta @ self.W[k + 1].T) * d_act
                    grads_W[k] = activations[k].T @ delta / Bsz
                    grads_b[k] = delta.mean(axis=0)

                if self.l2 > 0:
                    for k in range(len(self.W)):
                        grads_W[k] += self.l2 * self.W[k]

                # All gradients use the weights from the same forward pass.
                for k in range(len(self.W)):
                    self.W[k] -= self.lr * grads_W[k]
                    self.b[k] -= self.lr * grads_b[k]

            new_loss = self.loss(X, y)
            if verbose:
                print(f"Epoch {ep:3d} | loss {base_loss:.5f} -> {new_loss:.5
```

Choosing hidden activations

```
    def _act(self, z, last=False):
        if last:
            return sigmoid(z)
        return relu(z) if self.hidden_activation == "relu" else
sigmoid(z)

    def _act_prime(self, z, last=False):
        if last:
            return sigmoid_prime(z)
        return relu_prime(z) if self.hidden_activation == "relu"
else sigmoid_prime(z)
```

The output remains sigmoid for binary classification. ReLU is an option only for hidden units. Lecture 13 called `sigmoid` and `sigmoid_prime` directly. Here the helpers `_act` and `_act_prime` select the activation and its derivative, and the flag `last` identifies the output layer. The forward and cached-forward methods now call `_act`; the backward loop calls `_act_prime`. Its multiplication by `W_next.T` and its averaging over examples are unchanged.

Learning-rate decay

At epoch $t = 1, 2, \ldots$, with decay factor $0 < \gamma < 1$:

$$\alpha_t = \alpha_1 \gamma^{t-1}.$$

```
if self.lr_decay is not None:
    self.lr = self.base_lr * self.lr_decay ** (ep - 1)
```

The first epoch uses the initial learning rate; subsequent epochs use progressively smaller steps. With `lr_decay=None`, the learning rate stays constant. Decay changes the optimizer's step size, not the derivative formulas or the objective. It is different from L2 regularization and from weight decay. Reducing the learning rate too quickly can prevent useful progress; it does not repair vanishing gradients.

L2 in the objective and gradient

$$J_{\text{reg}} = J_{\text{BCE}} + \frac{\lambda}{2} \sum_{l=1}^{L} \|W^{(l)}\|_F^2.$$

```
base += 0.5 * self.l2 * sum((W ** 2).sum() for W in self.W)

# After computing the mean data-loss gradients:
grads_W[k] += self.l2 * self.W[k]
```

The coefficient `l2` is λ. The data loss is averaged over examples; the penalty is added separately, with no additional division by dataset or mini-batch size. Consequently, its derivative is λW in every batch. This keeps the reported objective and the update's gradient consistent. Biases are not penalized in this implementation.

The Frobenius norm squared is simply the sum of squared entries of a weight matrix. L2 discourages large weights; it does not change the local activation derivatives or repair gradient propagation directly. The later regularization section develops the generalization motivation and other methods.

Same API, extended model

In [5]:
```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

X, y = make_circles(n_samples=200, factor=0.5, noise=0.08, random_state=1)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0, stratify=y)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = SimpleMLP([2, 4, 4, 1], lr=0.3, seed=42,
                  hidden_activation="relu", l2=0.001, lr_decay=0.995)
model.train(X_train, y_train, epochs=500, batch_size=32, verbose=False)
```

```
print("Train acc:", accuracy_score(y_train, model.predict(X_train)))
print(" Test acc:", accuracy_score(y_test, model.predict(X_test)))
```

```
Train acc: 1.0
 Test acc: 1.0
```

This uses the same data, split, preprocessing, and architecture as the `SimplerMLP` example. It demonstrates how to enable the additional options. Because the activation, training duration, penalty, and schedule differ, these two runs are illustrative workflows rather than a controlled claim that one setting is universally better. Both constructors' seeds control weights and shuffling. All gradients are still collected before updates.

Output Layers

Output Layer: Regression Task

- **# of output neurons**:
  - 1 per dimension
- **Output layer activation function**:
  - None, *ReLU/softplus*, if positive, *sigmoid/tanh*, if bounded
- **Loss function**:
  - MeanSquaredError

In an object detection problem, determining the **bounding box** exemplifies a regression task where the output is multidimensional.

How does one select an appropriate activation function?

In a feed-forward network, activation functions in the hidden layers are chosen mainly for their impact on performance and convergence, while the activation function in the output layer is selected according to the requirements of the task.

- **Hidden layers:**

  The activation function (e.g., ReLU, GELU, tanh) mainly affects *training dynamics* — such as gradient flow, nonlinearity strength, and convergence rate. The choice is typically empirical, guided by performance, numerical stability, and optimization behavior.

- **Output layer:**

  The activation function is determined by the **nature of the prediction task**:

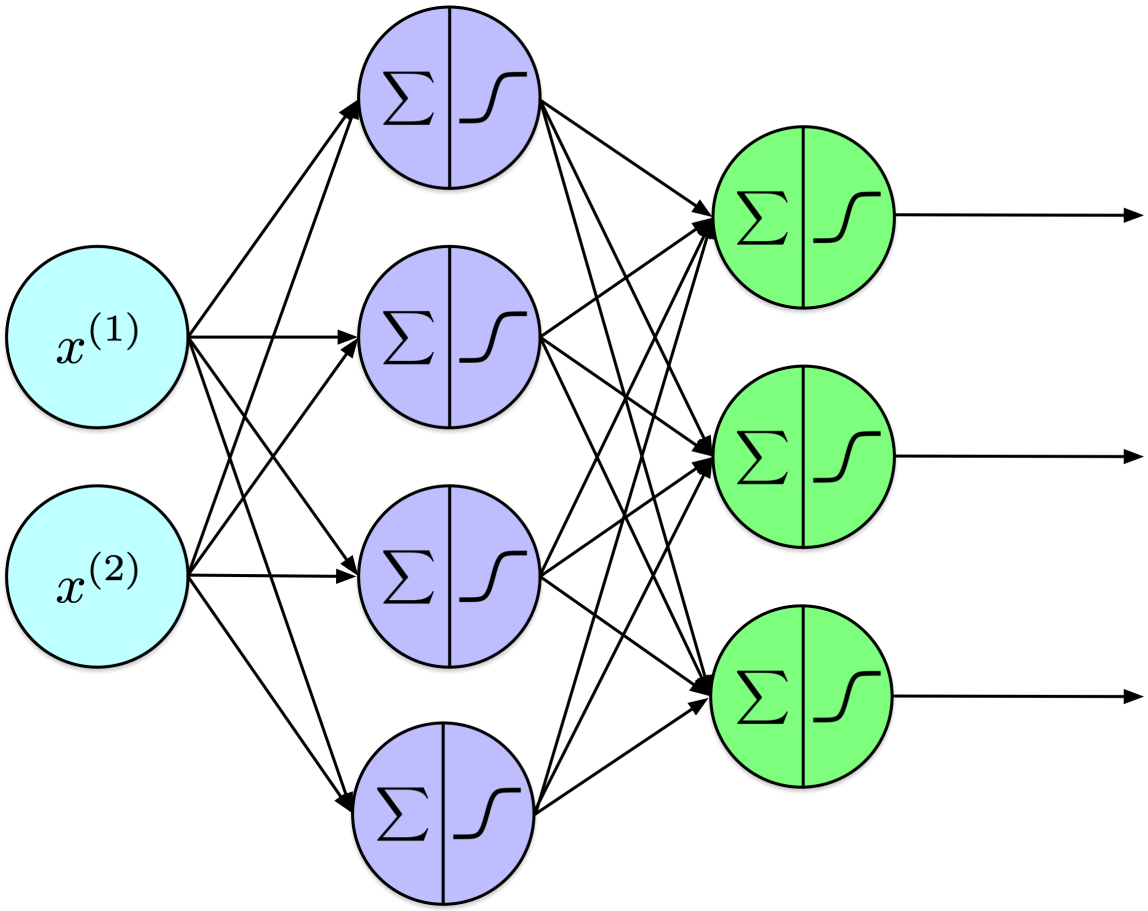
  - *Regression*: linear, ReLU, or softplus (depending on range constraints)
  - *Binary classification*: sigmoid
  - *Multi-class classification*: softmax

- *Multi-label classification*: sigmoid (per output unit)

Examples of **multi-output regression problems** (2–4 outputs) include:

- **2 outputs:**
  - Predicting both the **latitude and longitude** of a location from an image.
  - Estimating the **width and height** of a bounding box in object detection.
  - Forecasting **temperature and humidity** simultaneously.
- **3 outputs:**
  - Predicting the **x, y, z coordinates** of a point in space (e.g., hand-joint localization).
  - Estimating **RGB color values** from spectral measurements.
- **4 outputs:**
  - Predicting **bounding box coordinates** — $(x_{min}, y_{min}, x_{max}, y_{max})$ — in object detection.

Output Layer: Multi-Label



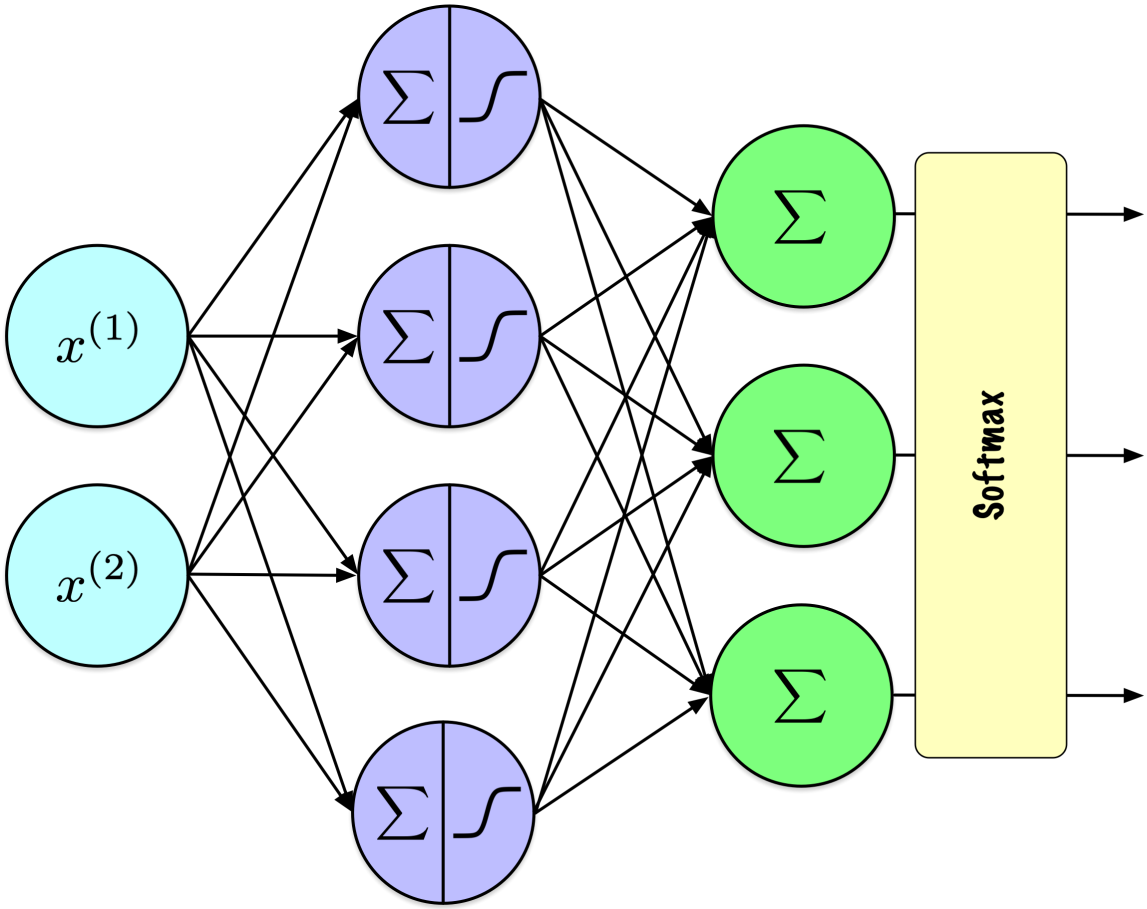
With the sigmoid activation function, this network makes multi-label predictions.

Why do independent sigmoid outputs not directly provide a probability distribution over mutually exclusive classes? Each output is a probability, but their sum is not constrained to equal one. Such outputs remain appropriate for multi-label classification.

Output Layer: Classification Task

- **# of output neurons**:
  - 1 if binary, 1 per class, if multi-label or multiclass.
- **Output layer activation function**:
  - *sigmoid*, if binary or multi-label, *softmax* if multi-class.
- **Loss function**:
  - cross-entropy

Softmax



Softmax ensures that all activation outputs fall between 0 and 1 and collectively sum to 1.

Observe that I have revised the representation of the output nodes to indicate that the softmax function is applied to the entire layer, rather than to individual nodes. This function transforms the raw output values of the layer into probabilities that sum to 1,

facilitating multi-class classification. This characteristic distinguishes it from activation functions like ReLU or sigmoid, which are typically applied independently to each node's output.

The argmax function is not suitable for optimization via gradient-based methods because it is piecewise constant, with zero derivatives away from ties and undefined derivatives at relevant boundaries. In contrast, the softmax function offers both a probabilistic interpretation and a computable derivative, making it more effective for such applications.

The argmax function can be applied *a posteriori* to trained networks for class prediction.

Softmax

The **softmax** function is an **activation function** used in **multi-class classification problems** to convert a vector of raw scores into **probabilities that sum to 1**.

Given a vector $\mathbf{z} = [z_1, z_2, \ldots, z_n]$:

$$\hat{p}_i = \mathrm{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

where $\hat{p}_i$ is the probability of the i-th class, and n is the number of classes.

Softmax emphasizes higher scores while suppressing lower ones, enabling a probabilistic interpretation of the outputs.

We clearly see that such an activation applies for an entire layer since the denominator depends on the values of all the z_j, for $j = 1, \ldots, n$.

In the **softmax** function, exponentiation serves a crucial mathematical and conceptual role: it transforms arbitrary real-valued scores into **positive, normalized probabilities** while **amplifying differences** between inputs.

1. **Ensures positivity:**

   The exponential function guarantees all outputs are positive, a necessary property for valid probabilities.

2. **Magnifies relative differences:**

   Exponentiation is a *monotonic nonlinear* transformation that increases large values much more than small ones. If one logit is slightly larger, e^{z_i} becomes *disproportionately larger*, sharpening the distribution (useful for emphasizing the most likely class).

3. **Preserves ordering:**

   Since e^x is strictly increasing, the class with the highest logit before softmax remains the one with the highest probability after.

4. **Facilitates gradient-based learning:**

   The exponential smooths the transition between classes, providing differentiable outputs and stable gradients for optimization with cross-entropy.

A *logit* is a pre-activation value (z_i). Each neuron in the output layer (before activation) produces a logit z_i.

Softmax

z_1	z_2	z_3	$\hat{p}_1$	$\hat{p}_2$	$\hat{p}_3$	$\sum$
1.47	-0.39	0.22	**0.69**	0.11	0.20	1.00
5.00	**6.00**	4.00	0.24	**0.67**	0.09	1.00
0.90	0.80	**1.10**	0.32	0.29	**0.39**	1.00
-2.00	**2.00**	-3.00	0.02	**0.98**	0.01	1.00

Softmax values for a vector of length 3.

1. **Maintains Relative Order**: The softmax function preserves the relative order of the input values. If one input is greater than another, its corresponding output will also be greater.

2. **Interpreted as probabilities**: Each value is in the range 0 to 1. The output values from the softmax function are normalized to sum to one, which allows them to be interpreted as probabilities.

3. **Relative Differences**: When the relative differences among the input values are small, the differences in the output probabilities remain small, reflecting the input distribution. When the input values are identical, the output values will be $\frac{1}{n}$, where n is the number of classes.

4. **Wide Range of Values**: The softmax function can effectively handle a wide range of input values, thanks to the exponential function and normalization, which scale the inputs to a probabilistic range.

These properties make the softmax function particularly useful for multi-class classification tasks in machine learning.

Softmax

https://www.youtube.com/watch?v=KpKog-L9veg

Cross-entropy loss function

The **cross-entropy** in a **multi-class classification task** for one example:

$$\ell(W) = -\sum_{k=1}^{K} y_k \log(\hat{p}_k)$$

Where:

- K is the **number of classes**.
- y_k is the **true distribution** for the class k.
- $\hat{p}_k$ is the **predicted probability** of class k from the model.
- The target vector y is expressed as a one-hot encoded vector of length K, where the element corresponding to the true class is set to 1, and all other elements are 0.
- Consequently, in the summation over classes, only the term associated with the true class contributes a non-zero value.
- Therefore, the cross-entropy loss for a single example is given by $-\log(\hat{p}_k)$, where $\hat{p}_k$ is the predicted probability for the true class.
- The predicted probability $\hat{p}_k$ is derived from the softmax function applied in the output layer of the neural network.

Cross-entropy loss function

- **Classification Problem**: 3 classes
  - Versicolour, Setosa, Virginica.
- **One-Hot Encoding**:
  - Setosa = $[0, 1, 0]$.
- **Softmax Outputs & Loss**:
  - $[0.22, \mathbf{0.7}, 0.08]$: Loss = $-\log(0.7) = 0.3567$.
  - $[0.7, \mathbf{0.22}, 0.08]$: Loss = $-\log(0.22) = 1.5141$.
  - $[0.7, \mathbf{0.08}, 0.22]$: Loss = $-\log(0.08) = 2.5257$.

Among the softmax outputs, cross-entropy evaluates only the component corresponding to $k = 2$ (Setosa), as the other entries in the one-hot encoded vector are zero. Here mathematical class indices run from 1 to 3. This relevant element is highlighted in bold. A higher probability for the true class produces a smaller loss; the three examples give losses of 0.3567, 1.5141, and 2.5257.

Case: one example

In [6]:
```python
import numpy as np
import matplotlib.pyplot as plt

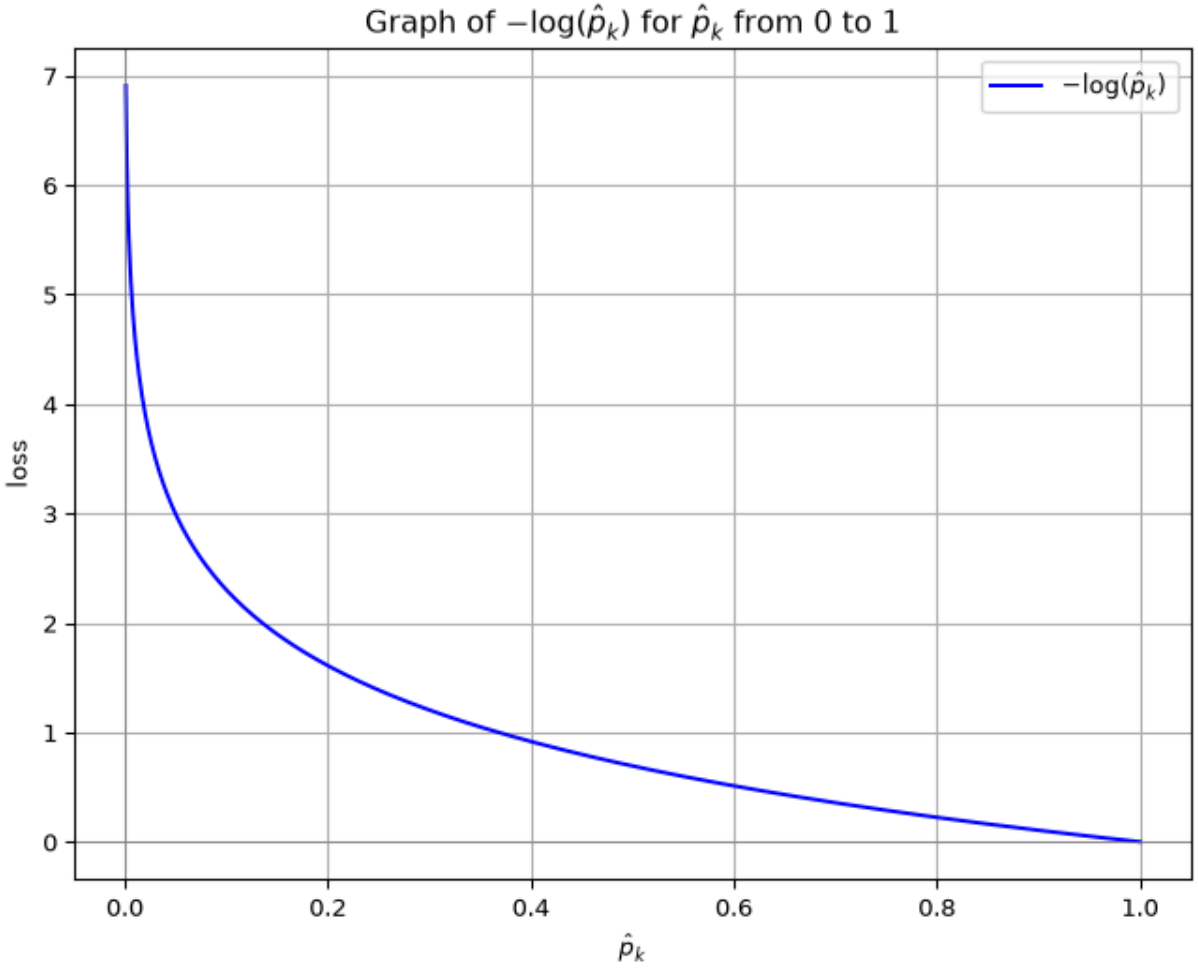
# Generate an array of p values from just above 0 to 1
p_values = np.linspace(0.001, 1, 1000)

# Compute the natural logarithm of each p value
ln_p_values = - np.log(p_values)

# Plot the graph
plt.figure(figsize=(8, 6))
plt.plot(p_values, ln_p_values, label=r'$-\log(\hat p_k)$', color='b')

# Add labels and title
plt.xlabel(r'$\hat p_k$')
plt.ylabel(r'loss')
plt.title(r'Graph of $-\log(\hat p_k)$ for $\hat p_k$ from 0 to 1')
plt.grid(True)
plt.axhline(0, color='gray', lw=0.5)  # Add horizontal line at y=0
plt.axvline(0, color='gray', lw=0.5)  # Add vertical line at x=0

# Display the plot
plt.legend()
plt.show()
```



- In the summation, only the term where $y_k = 1$ contributes a non-zero value.
- Due to the negative sign preceding the summation, the value of the function is $-\log(\hat{p}_k)$.
- If the predicted probability $\hat{p}_k$ is near 1, the loss approaches zero, indicating minimal penalty.
- Conversely, as $\hat{p}_k$ nears 0, indicating an incorrect prediction, the loss approaches infinity. This strongly penalizes assigning very low probability to the true class; it does not by itself guarantee faster convergence.

Case: dataset

For a dataset with N examples, the **average cross-entropy loss** over all examples is computed as:

$$J(W) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{i,k}\log(\hat{p}_{i,k})$$

Where:

- i indexes over the different **examples** in the dataset.
- $y_{i,k}$ and $\hat{p}_{i,k}$ are the **true** and **predicted probabilities** for class k of example i, respectively.

Regularization

Definition

Regularization comprises a set of techniques designed to enhance a model's ability to generalize by **mitigating overfitting**. By **discouraging excessive model complexity**, these methods **improve** the model's **robustness** and **performance** on unseen data.

Adding penalty terms to the loss

- In numerical optimization, it is standard practice to **incorporate additional terms** into the **objective function** to **deter undesirable model characteristics**.
- For a **minimization problem**, the optimization process aims to **circumvent the substantial costs** associated with these **penalty terms**.

Loss function

Consider the **mean absolute error** loss function:

$$\mathrm{MAE}(X, W) = \frac{1}{N} \sum_{i=1}^{N} |h_W(x_i) - y_i|$$

Where:

- W are the weights of our network.
- $h_W(x_i)$ is the output of the network for example i.
- y_i is the true label for example i.

Penalty term(s)

One or more terms can be added to the loss:

$$J_{\mathrm{reg}}(X, W) = \mathrm{MAE}(X, W) + \mathrm{penalty}$$

Norm

A **norm** assigns a non-negative length to a vector.

The ℓ_p **norm** of a vector $\mathbf{z} = [z_1, z_2, \ldots, z_n]$ is defined as:

$$\|\mathbf{z}\|_p = \left(\sum_{i=1}^{n} |z_i|^p \right)^{1/p}$$

With larger p, the ℓ_p norm increasingly highlights larger z_i values due to exponentiation.

A **norm** is a function that assigns a non-negative length or size to each vector in a vector space, satisfying certain properties: positivity, scalar multiplication, the triangle inequality, and the property that the norm is zero if and only if the vector is zero.

ℓ_1 and ℓ_2 norms

The ℓ_1 **norm** (**Manhattan norm**) is:

$$\|\mathbf{z}\|_1 = \sum_{i=1}^{n} |z_i|$$

The ℓ_2 **norm** (**Euclidean norm**) is:

$$\|\mathbf{z}\|_2 = \sqrt{\sum_{i=1}^{n} z_i^2}$$

ℓ_1 and ℓ_2 regularization

Let $\mathbf{w}$ collect all weight entries. The nonnegative coefficients λ_1 and λ_2 control the penalties; zero disables the corresponding term.

$$J_{\text{reg}} = \text{MAE}(X, W) + \lambda_1 \|\mathbf{w}\|_1 + \frac{\lambda_2}{2} \|\mathbf{w}\|_2^2.$$

The L1 term sums absolute weight values. The usual L2 penalty sums squared weight values; it uses the squared Euclidean or Frobenius norm, rather than the unsquared norm. The factor one-half is a convention that makes its derivative $\lambda_2 W$, matching `SimpleMLP` above. Coefficient conventions differ across libraries.

Guidelines

- ℓ_1 **Regularization**:
  - Promotes **sparsity**, setting many weights to zero.
  - Useful for **feature selection** by reducing feature reliance.

- ℓ_2 **Regularization**:
  - Promotes **small, distributed weights** for stability.
  - Ideal when **all features contribute** and reducing complexity is key.

See kong_and_yu-2018 for an example.

Keras example

```
import tensorflow as tf
from tensorflow.keras.layers import Dense

regularizer = tf.keras.regularizers.l2(0.001)

dense = Dense(50, kernel_regularizer=regularizer)
```

This layer specifically utilizes ℓ_2 regularization, in contrast to the prior discussion where regularization was applied globally across the entire model.

Dropout

Dropout is a regularization technique in neural networks where *randomly selected neurons are ignored during training*, reducing overfitting by **preventing co-adaptation of features**.

Hinton et al. (2012)

Dropout

- During each training step, each neuron in a dropout layer has a probability p of being **excluded from the computation**, typical values for p are between 10% and 50%.
- While seemingly counterintuitive, this approach **prevents the network from depending on specific neurons**, promoting the **distribution of learned representations across multiple neurons**.

Dropout

- Dropout is one of the **most popular** and **effective** methods for reducing overfitting.
- The typical improvement in performance is **modest**, usually around 1 to 2%.

During training, each neuron in a dropout layer is randomly excluded (set to zero) with a probability p—the dropout rate.

When dropout "sets a neuron to zero," it means that for the current training step (mini-batch), that neuron's output activation is replaced by zero. The neuron still exists in the

network — its weights are not removed or altered — but its output contributes nothing to the forward or backward pass for that batch.

In the next batch, a new random mask is sampled, so a different subset of neurons is dropped. Over many iterations, all neurons participate, but never all at once.

At inference time, dropout is disabled—all neurons are active, and their outputs are not scaled.

Keras

```
import keras
from keras.models import Sequential
from keras.layers import InputLayer, Dropout, Flatten, Dense

model = keras.Sequential([
    InputLayer(shape=[28, 28]),
    Flatten(),
    Dropout(rate=0.2),
    Dense(300, activation="relu"),
    Dropout(rate=0.2),
    Dense(100, activation="relu"),
    Dropout(rate=0.2),
    Dense(10, activation="softmax")
])
```

Adding `Dropout` layers to the Fashion-MNIST model from Lecture 11.

The dropout rate may differ between layers; larger rates can be applied to larger layers, while smaller rates are suitable for smaller layers. It is common practice in many networks to apply dropout only after the final hidden layer.

Definition

Early stopping is a regularization technique that halts training once the model's performance on a validation set begins to degrade, preventing overfitting by stopping before the model learns noise.

Geoffrey Hinton calls this the *"beautiful free lunch."*

Early Stopping

In [7]:
```
from copy import deepcopy
from sklearn.metrics import root_mean_squared_error
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import SGDRegressor
```

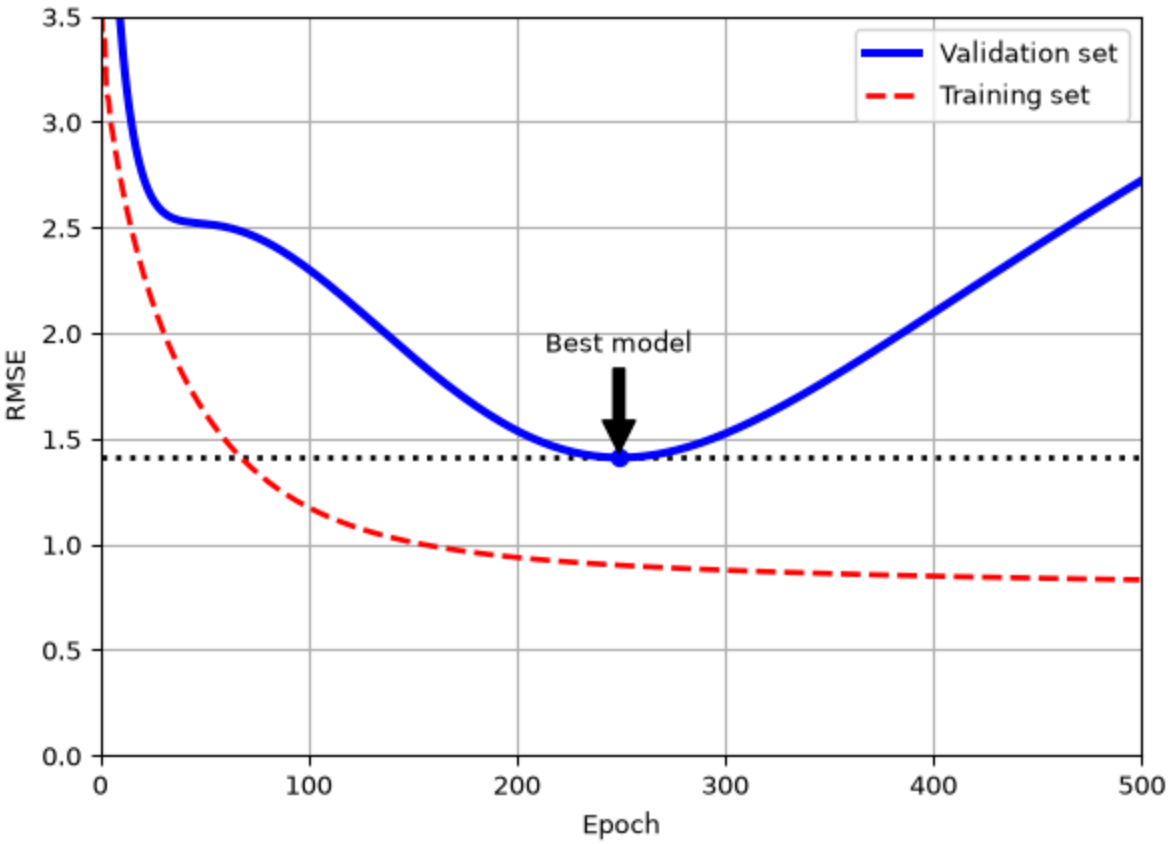
```
# extra code – creates the same quadratic dataset as earlier and splits it
np.random.seed(42)
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X ** 2 + X + 2 + np.random.randn(m, 1)
X_train, y_train = X[: m // 2], y[: m // 2, 0]
X_valid, y_valid = X[m // 2 :], y[m // 2 :, 0]

preprocessing = make_pipeline(PolynomialFeatures(degree=90, include_bias=Fal
                              StandardScaler())
X_train_prep = preprocessing.fit_transform(X_train)
X_valid_prep = preprocessing.transform(X_valid)
sgd_reg = SGDRegressor(penalty=None, eta0=0.002, random_state=42)
n_epochs = 500
best_valid_rmse = float('inf')
train_errors, val_errors = [], []  # extra code – it's for the figure below

for epoch in range(n_epochs):
    sgd_reg.partial_fit(X_train_prep, y_train)
    y_valid_predict = sgd_reg.predict(X_valid_prep)
    val_error = root_mean_squared_error(y_valid, y_valid_predict)
    if val_error < best_valid_rmse:
        best_valid_rmse = val_error
        best_model = deepcopy(sgd_reg)

    # extra code – we evaluate the train error and save it for the figure
    y_train_predict = sgd_reg.predict(X_train_prep)
    train_error = root_mean_squared_error(y_train, y_train_predict)
    val_errors.append(val_error)
    train_errors.append(train_error)

# extra code – this section generates and saves Figure 4–20
best_epoch = np.argmin(val_errors)
plt.annotate('Best model',
             xy=(best_epoch, best_valid_rmse),
             xytext=(best_epoch, best_valid_rmse + 0.5),
             ha="center",
             arrowprops=dict(facecolor='black', shrink=0.05))
plt.plot([0, n_epochs], [best_valid_rmse, best_valid_rmse], "k:", linewidth=
plt.plot(val_errors, "b-", linewidth=3, label="Validation set")
plt.plot(best_epoch, best_valid_rmse, "bo")
plt.plot(train_errors, "r--", linewidth=2, label="Training set")
plt.legend(loc="upper right")
plt.xlabel("Epoch")
plt.ylabel("RMSE")
plt.axis([0, n_epochs, 0, 3.5])
plt.grid()
plt.show()
```



Attribution: Géron (2022), 04_training_linear_models.ipynb.

Prologue

Summary

- **Gradient propagation:** activation derivatives and initialization affect training; ReLU and suitable scaling address some limitations of sigmoid networks.
- **Implementation:** `SimpleMLP` retains the `SimplerMLP` structure and adds hidden activations, optional L2, and learning-rate decay.
- **Loss Functions:**
  - **Regression Tasks:** Mean Squared Error (MSE).
  - **Classification Tasks:** Cross-Entropy Loss with Softmax activation for multi-class outputs.
- **Regularization Techniques:**
  - **L1 and L2 Regularization:** Add penalty terms to the loss to discourage large weights.
  - **Dropout:** Randomly deactivate neurons during training to prevent overfitting.
  - **Early Stopping:** Halt training when validation performance deteriorates.

Next lecture

- We will introduce convolutional neural networks.

References

Géron, Aurélien. 2022. *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3rd ed. O'Reilly Media, Inc.

Glorot, Xavier, and Yoshua Bengio. 2010. "Understanding the Difficulty of Training Deep Feedforward Neural Networks." In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, edited by Yee Whye Teh and Mike Titterington, vol. 9. Proceedings of Machine Learning Research. PMLR. https://proceedings.mlr.press/v9/glorot10a.html.

Hinton, Geoffrey E., Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2012. "Improving Neural Networks by Preventing Co-Adaptation of Feature Detectors." *CoRR* abs/1207.0580. http://arxiv.org/abs/1207.0580.

Russell, Stuart, and Peter Norvig. 2020. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson. http://aima.cs.berkeley.edu/.

Marcel **Turcotte**

Marcel.Turcotte@uOttawa.ca

School of Electrical Engineering and **Computer Science** (EE**CS**)

University of Ottawa